

Supply Chain Compromise Runbook: SCA and IDE Extensions

Last updated: 2026-06-04

Purpose

This runbook helps the SOC investigate and respond to suspected developer supply chain compromise, including malicious open source packages identified by software composition analysis (SCA), malicious package manager activity, malicious IDE extensions, and related developer-tool attacks.

Use this runbook when an alert, threat advisory, developer report, SCA finding, package scan, EDR event, proxy log, or repository audit indicates that a developer workstation, CI/CD runner, build container, package cache, IDE, or code repository may have executed malicious package or extension code.

Scope

In scope:

- npm, pnpm, Yarn, and Node.js package install or update activity.
- Software composition analysis findings for known malicious packages, transitive dependencies, or remote package sources.
- VSCode, Cursor, Windsurf, and other IDE extension installs or auto-updates.
- Open VSX, Visual Studio Marketplace, and other IDE extension marketplace compromise.
- Developer endpoints, privileged admin workstations, build agents, CI/CD runners, artifact repositories, source repositories, and package publishing workflows.
- Credentials accessible to the affected environment, including GitHub, Bitbucket, npm, cloud, SSH, Vault, Kubernetes, 1Password CLI sessions, environment variables, and CI/CD secrets.

Out of scope:

- Normal vulnerable dependency remediation where there is no evidence of malware execution.
- Browser extension-only incidents unless they are part of the developer compromise chain.

Severity

Treat the incident as **Critical** if any of the following are true:

- A known malicious package, transitive dependency, remote dependency, or IDE extension version was installed or auto-updated.
- The host or CI runner contacted known command-and-control infrastructure.
- Malware persistence artifacts are present.
- npm package publish tokens, GitHub tokens, cloud credentials, SSH keys, Vault tokens, or CI/CD secrets were present on the affected system.
- A repository was made public, cloned unexpectedly, had a suspicious workflow committed, or had a suspicious package version published.
- The affected system had access to production, sensitive data, package publishing, repository administration, or CI/CD administration.

Treat as **High** if exposure is suspected but not confirmed. Downgrade only after AppSec and Incident Response agree that the package or extension never executed and no credentials were exposed.

Roles

- SOC: Triage alerts, preserve evidence, run hunts, coordinate containment, track timeline.
- AppSec: Validate package/extension exposure, inspect manifests and lockfiles, advise dependency remediation.

- Developer Platform / DevOps: Disable affected pipelines, revoke build credentials, purge package caches, rebuild runners.
- Endpoint / IT: Isolate developer workstations, collect forensic artifacts, rebuild endpoints when required.
- Cloud / IAM: Revoke and rotate cloud credentials, review suspicious API use, validate privilege boundaries.
- Legal / Privacy / Communications: Engage if source code, customer data, regulated data, or third-party systems may be exposed.

Initial Triage

Start an incident if any trigger below is observed:

- Alert for a known malicious package, malicious package version, malicious transitive dependency, malicious remote dependency, or malicious IDE extension.
- `npm`, `pnpm`, `yarn`, `node`, `bun`, `code`, `cursor`, or an extension host process spawns shell, PowerShell, Python, curl, wget, osascript, or another scripting utility.
- Unexpected outbound network activity from a package install, build step, VSCode extension host, or CI runner.
- Package manifests or lockfiles reference an HTTP URL dependency, unexpected package, or package version listed in this runbook.
- A developer reports unusual install prompts, fake progress messages, a sudo/admin password prompt during package install, or VSCode behaving oddly after an extension update.
- GitHub, Bitbucket, npm, or cloud audit logs show unusual token use, repo creation, repo visibility changes, package publishing, workflow edits, or secrets access.

Capture the following immediately:

- Alert name, timestamp, source system, affected user, affected host, and affected repository or pipeline.
- Package manager command, package name, package version, extension ID, extension version, and install/update timestamp.
- Whether the affected system is a developer workstation, CI/CD runner, build container, production host, or package publishing environment.
- All credentials that may have been accessible on disk, in environment variables, in secret stores, or through active CLI sessions.

Tactical Response Order

Credential containment comes before eradication. Rebuilding or cleaning a host while a stolen token remains valid can leave the attacker with durable access outside the affected machine.

Upon detection:

1. Isolate the affected machine or runner from the network.
2. Pause affected CI/CD jobs, runner pools, package publishing workflows, and automated deployment paths.
3. Determine exactly which secrets the affected environment could access.
4. Map the transitive secret exposure graph: for each exposed credential, identify the repositories, clouds, CI/CD systems, package registries, secret stores, databases, clusters, SaaS applications, and additional secrets it could read, mint, modify, or administer.
5. Revoke or rotate all directly and transitively exposed secrets, prioritizing credentials that can access source control, cloud control planes, production data, secret stores, package publishing, CI/CD administration, or other secrets.
6. Validate that old credentials, sessions, refresh tokens, deploy keys, webhooks, OAuth grants, and service principal credentials no longer work.
7. Shift to eradication, rebuild, and controlled reintroduction of the affected machine or runner.

Do not treat host cleanup as complete until the credential exposure graph is closed. Missing one token, cloud key, or secret-store credential can preserve compromise and allow later data theft, tampering, destructive activity, or ransom activity.

First 30 Minutes

1. Open an incident and assign an incident commander.
2. Isolate affected developer endpoints from the network using EDR.
3. Pause affected CI/CD jobs, runners, and package publishing workflows.
4. Preserve evidence where doing so does not delay isolation or credential containment.
5. Build the credential exposure graph for the affected user, host, runner, repository, and pipeline.
6. Disable, revoke, or rotate all directly and transitively exposed npm, GitHub, Bitbucket, GitLab, cloud, Vault, SSH, SaaS, and CI/CD credentials.
7. Prioritize immediate rotation for credentials that can access source control, cloud control planes, production data, secret stores, package publishing, CI/CD administration, or other secrets.
8. Block known malicious domains, IPs, and package artifacts at DNS, proxy, firewall, EDR, and artifact repository controls.
9. Notify AppSec, Developer Platform, Endpoint, Cloud/IAM, and secret-store owners.
10. Search for the same package, extension, or IoCs across all developer endpoints, repositories, build logs, and package caches.

Do not run `npm install`, `pnpm install`, `yarn install`, or reopen the affected workspace on a suspected host until evidence has been collected. Re-running install scripts can re-execute malware.

Containment

Endpoint Containment

- Isolate affected workstations and CI runners.
- Collect volatile process, network, and logged-in user context where tooling allows.
- Preserve the following before deletion:
 - Package manifests and lockfiles.
 - `node_modules` path for the suspicious package.
 - npm, pnpm, and Yarn caches.
 - VSCode extension directories.
 - Shell history and terminal logs when available.
 - EDR process tree and network telemetry.
 - Relevant browser downloads if the package came from a copied command or AI-generated instruction.

CI/CD Containment

- Disable affected workflow jobs and runner pools.
- Remove affected runners from service. Prefer rebuilding runners from a known-good image.
- Stop package publishing automation until publish credentials are rotated and the pipeline is reviewed.
- Review CI caches for malicious dependencies, poisoned build artifacts, and cached extension or package tarballs.
- Purge package manager caches after evidence is captured.

Source Control Containment

- Revoke suspicious GitHub/Bitbucket/GitLab sessions and tokens.
- Review recent repository activity for:
 - Newly created repositories.
 - Repositories changed from private to public.
 - Unexpected forks.
 - Unexpected workflow files in `.github/workflows/`.
 - Unexpected commits to branches such as `oidc-*`, `chore/add-codeql-static-analysis`, or similarly benign-looking branches.
 - New deploy keys, SSH keys, OAuth apps, GitHub Apps, webhooks, and Actions secrets.
- Temporarily restrict package publishing and workflow write permissions for affected maintainers.

Package Source Containment

- Block direct public npm downloads if an approved package proxy or artifact manager exists.
- Purge malicious package versions from internal mirrors, proxies, and package caches.
- Pin or force safe versions through the approved package source.
- Disable or restrict package install lifecycle scripts in CI where practical.
- Block package dependencies that resolve from arbitrary HTTP URLs unless explicitly approved.

Investigation

Determine Exposure

For each affected package or extension, determine:

- Was it listed in `package.json`, `package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`, `npm-shrinkwrap.json`, or a generated SBOM?
- Was it present in `node_modules`, a package cache, a build image, or a runner cache?
- Did the package install script execute?
- Was the installation performed on a workstation, CI runner, container build, or production-like system?
- Which user account ran the install?
- Which credentials were accessible to that user and host?
- Which additional systems, secrets, roles, tokens, keys, repositories, packages, and cloud resources those credentials could access or administer?
- Did the host contact known malicious infrastructure?
- Did the host create or modify repositories, workflows, package versions, cloud resources, or secret stores afterward?

Build a credential exposure graph that includes:

- Local secrets on disk, in shell history, in editor configuration, in package manager configuration, in environment variables, and in active CLI sessions.
- Source control access: PATs, SSH keys, deploy keys, GitHub CLI sessions, OAuth grants, GitHub Apps, repository webhooks, Actions secrets, and workflow permissions.
- Package registry access: npm tokens, publish automation, trusted publishing configuration, package maintainer sessions, and artifact repository credentials.
- Cloud access: access keys, refresh tokens, service principals, managed identities, workload identity/OIDC trust paths, role assumptions, and metadata-service credentials.
- Secret-store access: Vault, cloud secret managers, Kubernetes secrets, CI/CD secret stores, 1Password CLI sessions, and any credential that can read or mint more credentials.
- Downstream blast radius: production systems, sensitive data stores, package publishing authority, infrastructure administration, and any path that could enable persistence or privilege expansion.

Host Forensics

Look for:

- Suspicious child processes of `npm`, `pnpm`, `yarn`, `node`, `bun`, `code`, `cursor`, or VSCode extension host processes.
- Install scripts invoking `curl`, `wget`, `powershell`, `pwsh`, `cmd`, `bash`, `sh`, `python`, `osascript`, `chmod`, `nohup`, `bun`, or encoded commands.
- Unexpected files in `/tmp`, `/var/tmp`, `%TEMP%`, `%PROGRAMDATA%`, and user profile directories.
- New persistence through LaunchAgents, systemd user services, scheduled tasks, registry autoruns, shell profile changes, VSCode tasks, or Claude Code hooks.
- Unexpected access to `.npmrc`, `.git-credentials`, `~/.config/gh/hosts.yml`, SSH keys, `.env` files, Docker config, cloud config, Vault tokens, Kubernetes service account tokens, and 1Password CLI sessions.

Repository and Package Publishing Review

Review source control and package registry logs for:

- New package versions published by affected accounts.
- Publish activity using OIDC or trusted publishing during or shortly after the compromise window.
- Provenance attestations that are technically valid but tied to unexpected commits, branches, or workflow runs.
- Suspicious orphan commits or hidden GitHub-hosted payloads.
- New workflows that request broad permissions, especially `id-token: write`, `contents: write`, `actions: write`, or secrets access.
- Public repositories with campaign-specific descriptions listed in this runbook.

Cloud and Secret Store Review

For any host with cloud or secret access, review:

- AWS STS, Secrets Manager, SSM Parameter Store, ECR, IAM, and metadata-service access.
- Azure CLI logins, service principal usage, Key Vault access, and token issuance.
- GCP service account usage, application-default credentials, Secret Manager access, and unusual API user agents.
- Vault token lookup, secret reads, and token creation.
- Kubernetes service account token usage, secret reads, and API calls from unusual IPs.

Hunting Queries and Search Patterns

Use the examples below as starting points and adapt to available telemetry.

Endpoint Process Hunts

Search for package managers or editors spawning scripting tools:

```
Parent process in (npm, npm.exe, pnpm, pnpm.exe, yarn, yarn.exe, node, node.exe, bun, bun.exe,
code, Code.exe, cursor, Cursor.exe)
AND child process in (curl, wget, powershell, pwsh, cmd, bash, sh, python, python3, osascript,
chmod, nohup)
```

Search for known suspicious command fragments:

```
plain-crypto-js
node setup.js
sfrclak.com
packages.npm.org/product
/tmp/ld.py
6202033.ps1
6202033.vbs
com.apple.act.mond
firedalazer
m-kosche.com
kitty/cat.py
```

Lockfile and Manifest Hunts

Search all repositories for malicious package names, versions, and URL dependencies:

```
rg -n 'axios@?1\.14\.1|axios@?0\.30\.4|plain-crypto-js|nrwl\.angular-console|react-state-optimizer-core|coinbase-desktop-sdk|packages\.storeartifact|npm\.jpartifacts|package\.storeartifacts|npm\.artifactsnpm|https?:\/\/S+\.tgz|https?:\/\/S+\/npm/' .
```

Search for package lifecycle scripts:

```
rg -n '(preinstall|install|postinstall|prepare)|curl|wget|powershell|python|bun run|node setup\.js' package.json package-lock.json pnpm-lock.yaml yarn.lock npm-shrinkwrap.json
```

VSCode Extension Hunts

On endpoints, check installed extensions:

```
code --list-extensions --show-versions
cursor --list-extensions --show-versions
```

Search extension directories for known malicious versions and suspicious behavior:

```
rg -n "nrwl\.angular-console|18\.95\.0|firedalazer|kitty/cat\.py|curl|powershell|Invoke-WebRequest|child_process|execSync|https?://" "$env:USERPROFILE\.vscode\extensions"
"$env:USERPROFILE\.cursor\extensions"
```

Common extension paths:

- Windows: %USERPROFILE%\vscode\extensions, %USERPROFILE%\cursor\extensions
- macOS/Linux: ~/.vscode/extensions, ~/.cursor/extensions, ~/.windurf/extensions

GitHub and Repository Hunts

Search audit logs, repository metadata, and public GitHub exposure for:

```
Repository names: Shai-Hulud
Repository descriptions: Shai-Hulud Migration
Repository descriptions: A Mini Shai-Hulud has Appeared.
Repository descriptions: niagA oG eW ereH :duluH-iahS
Repository descriptions: Miasma: The Spreading Blight
Repository suffixes: -migration
Unexpected workflows: .github/workflows/shai-hulud-workflow.yml
Unexpected workflows: .github/workflows/discussion.yaml
Unexpected workflows: .github/workflows/codeql.yml
Suspicious branches: oidc-*
Suspicious branches: chore/add-codeql-static-analysis
```

Review GitHub audit logs for:

- Personal access token creation or use from unusual IPs.
- GitHub CLI OAuth token use from unusual IPs.
- Workflow file creation, update, or dispatch.

- Repository creation, clone, visibility change, fork, transfer, or branch protection change.
- npm trusted publishing or OIDC token exchange activity.
- New deploy keys, Actions secrets, webhooks, GitHub Apps, OAuth apps, and runner registrations.

Eradication

Preferred eradication is rebuild, not manual cleanup.

Begin eradication after isolation is complete and directly or transitively exposed credentials are revoked or actively being rotated.

- Reimage affected developer endpoints where malware execution is confirmed or likely.
- Rebuild affected CI runners from clean base images.
- Delete and recreate build containers and ephemeral environments.
- Remove malicious package versions and VSCode extensions.
- Purge npm, pnpm, Yarn, VSCode, Cursor, and CI caches after forensic preservation.
- Replace affected package versions with known-good versions from approved package sources.
- Review source repositories for malicious workflow changes, orphan commits, poisoned package manifests, and unauthorized publishing configuration.
- Remove unauthorized persistence artifacts and validate through EDR rescans.

For npm projects:

```
Remove-Item -Recurse -Force -LiteralPath .\node_modules  
npm cache clean --force
```

Then reinstall only from a clean lockfile and approved package source. Do not reuse a lockfile until AppSec confirms it does not reference compromised versions or URL-based dependencies.

Credential Rotation

Assume any credential reachable from the affected environment is exposed.

Rotate secrets by reachability, not just by where they were found. If an exposed token could read a secret store, assume the readable secrets are also exposed. If a cloud role could assume another role, read deployment outputs, access CI/CD variables, or retrieve database credentials, include those downstream secrets in scope.

Rotate or revoke:

- GitHub personal access tokens, GitHub CLI OAuth tokens, deploy keys, SSH keys, GitHub Apps, OAuth apps, and fine-grained tokens.
- npm access tokens, automation tokens, granular tokens, trusted publishing OIDC paths, and package maintainer sessions.
- Bitbucket/GitLab tokens and SSH keys.
- AWS access keys, STS sessions, IAM role trust paths, web identity tokens, and secrets retrieved by the affected host.
- Azure service principals, managed identity tokens, CLI sessions, and Key Vault secrets.
- GCP service account keys, application-default credentials, OAuth refresh tokens, and Secret Manager secrets.
- Vault tokens, Kubernetes service account tokens, Docker registry credentials, and CI/CD secrets.
- 1Password CLI sessions or any secrets accessible through an active **op** session.

Use this order when impact is broad:

1. Disable active attacker access: repository, package publishing, cloud admin, secret-store admin, and CI/CD admin tokens.
2. Rotate credentials that can read, mint, modify, or administer other secrets.

3. Rotate production and high-privilege secrets.
4. Rotate package publishing and developer platform credentials.
5. Rotate lower-privilege developer tokens and SSH keys.
6. Validate that old credentials, refresh tokens, sessions, deploy keys, webhooks, OAuth grants, and service principal credentials no longer work.

Recovery

Before returning systems to service:

- Confirm the malicious package or extension is removed and cannot be reinstalled from internal package sources.
- Confirm all relevant credentials have been rotated and old credentials are disabled.
- Confirm no suspicious repositories, workflows, branches, packages, webhooks, deploy keys, or OAuth apps remain.
- Confirm EDR, DNS, proxy, and source control logs show no continuing C2, exfiltration, or publishing activity.
- Re-enable CI/CD pipelines only after they run from clean runners and approved package sources.
- Ask affected developers to work from rebuilt machines or clean sandboxed environments.

Post-Incident Actions

- Add detections for package install lifecycle script abuse.
- Add detections for VSCode extension host network activity and suspicious child processes.
- Enforce approved package sources for developer and CI environments.
- Consider dependency cooldowns to prevent immediate adoption of newly published package versions.
- Prefer ephemeral CI runners and sandboxed developer environments.
- Disable long-lived package publish tokens where OIDC trusted publishing can be safely implemented.
- Restrict or review VSCode extension auto-update behavior for high-risk users.
- Maintain an approved extension allowlist for VSCode-compatible editors.
- Generate SBOMs and keep package manifests and lockfiles in source control.
- Review whether AI coding tools introduced suspicious or unnecessary dependencies.

Generic IoCs

NPM Malware

Host and process indicators:

- `npm`, `pnpm`, `yarn`, `node`, or `bun` spawning shell, PowerShell, Python, curl, wget, or `osascript`.
- Lifecycle scripts using `preinstall`, `install`, `postinstall`, or `prepare` to fetch remote code.
- `package.json`, lockfiles, or package tarballs referencing arbitrary HTTP URLs.
- Newly added dependencies that are not imported by application code.
- Obfuscated JavaScript using large encoded string arrays, runtime decoding, `eval()`, or hidden downloaders.
- Sudden creation of `.vscode/tasks.json`, `.claude/settings.json`, LaunchAgents, systemd user services, scheduled tasks, or shell profile modifications.
- Access to `.npmrc`, `.git-credentials`, `~/.config/gh/hosts.yml`, `.env`, SSH keys, cloud configs, Docker configs, Vault tokens, or Kubernetes service account tokens during install.

Repository and package indicators:

- Unexpected package versions published by internal maintainers.
- Valid package provenance tied to unexpected commits, branches, or workflow runs.
- Package publish events shortly after developer endpoint compromise.
- New package dependencies resolving from non-registry URLs.
- New public repositories containing secret dumps.

VSCode Malware

Host and process indicators:

- `code`, `cursor`, or extension host process spawning shell, PowerShell, Python, curl, wget, Node, or Bun.
- Extension activation immediately followed by outbound network activity.
- Extension packages with unexpected `activationEvents`, `extensionDependencies`, `extensionPack`, or obfuscated JavaScript.
- Extension updates during a published exposure window.
- Extension files that differ materially from the corresponding source repository release.

Repository and extension indicators:

- Extension ID and version in a threat advisory.
- Marketplace version with no matching source repository tag or release.
- Extension auto-update around the same time as suspicious token use.
- Extension package containing hidden payloads, orphan commit references, or shell command launchers.

Campaigns

Mini Shai-Hulud (2026)

Summary

Mini Shai-Hulud is an ongoing developer supply chain campaign first observed in April 2026. It targets npm, PyPI, Composer, GitHub Actions, and VSCode-related tooling. Public reporting links the campaign to TeamPCP with varying confidence depending on the wave. The malware commonly steals developer and CI/CD secrets, uses Bun to execute large obfuscated JavaScript payloads, exfiltrates through attacker-created GitHub repositories, and may install persistence.

IoCs

Package and execution indicators:

- Install-time or import-time hook downloads a Bun runtime.
- Large obfuscated JavaScript payloads such as `execution.js`, `router_runtime.js`, or root-level `index.js`.
- `package.json` lifecycle script such as `"preinstall": "bun run index.js"`.
- Compromised ecosystems include npm, PyPI, Composer, GitHub Actions, and VSCode/Open VSX.

Repository descriptions:

- `A Mini Shai-Hulud has Appeared.`
- `niagA oG eW ereH :duluH-iahS`

Host indicators:

- `~/.local/share/kitty/cat.py`
- `~/Library/LaunchAgents/com.user.kitty-monitor.plist`
- `~/.config/systemd/user/kitty-monitor.service`
- `/tmp/kitty-*`
- `/var/tmp/.gh_update_state`
- `%USERPROFILE%\local\share\kitty\cat.py`
- `%TEMP%\kitty-*`
- `%TEMP%\gh_update_state`
- `.claude/settings.json` SessionStart hooks
- `.vscode/tasks.json`

Network and C2 indicators:

- `api.github.com/search/commits?q=firedalazer`

- `firedalazer`
- `m-kosche[.]com`
- `185.95.159[.]32`
- `t[.]m-kosche[.]com:443/api/public/otel/v1/traces`

Known affected examples:

- `nrwl.angular-console@18.95.0`
- `actions-cool/issues-helper`
- `intercom-client@7.0.4`
- `intercom-client@7.0.5`
- `mbt@1.2.48`
- `@cap-js/db-service@2.10.1`
- `@cap-js/postgres@2.2.2`
- `@cap-js/sqlite@2.2.2`
- `lightning@2.6.2` and `lightning@2.6.3` on PyPI
- Multiple `@tanstack/*`, `@antv/*`, `@uipath/*`, `@mistralai/*`, `@opensearch-project/*`, `@draftauth/*`, and `@draftlab/*` packages reported in May 2026 waves.

Miasma (2026)

Summary

Miasma is a June 2026 supply chain compromise affecting packages under the `@redhat-cloud-services` npm namespace. Public reporting describes it as a Mini Shai-Hulud-derived or Mini Shai-Hulud-style payload. The malware used install-time execution, obfuscated JavaScript, and credential collection focused on developer, CI/CD, GitHub, npm, cloud, Kubernetes, and Vault environments.

IoCs

Repository and cloud indicators:

- Attacker-created repository description: `Miasma: The Spreading Blight`
- GCP query user-agent: `google-api-nodejs-client/7.0.0 gl-node/20.11.0 gccl/7.0.0`
- Suspicious Red Hat-related branches named `oidc-*`
- Obfuscated payloads such as `index.js` or `_index.js`
- Package lifecycle execution using `preinstall` to run `node index.js`
- Temporary payload and Bun runtime paths such as `/tmp/p*.js`, `/tmp/b-*/bun, %TEMP%\p*.js`, and `%TEMP%\b-*/bun.exe`
- GitHub Actions workflows requesting `id-token: write` and publishing packages with valid SLSA provenance from unexpected commits.

Microsoft published a newer package-version table on 2026-06-02 than the Wiz 2026-06-01 table. Use the Microsoft list below as the current source of truth for version matching, while retaining the Wiz-specific repository description and GCP user-agent indicators above.

Package	Compromised versions
<code>@redhat-cloud-services/types</code>	<code>3.6.1</code> , <code>3.6.2</code> , <code>3.6.4</code>
<code>@redhat-cloud-services/frontend-components-utilities</code>	<code>7.4.1</code> , <code>7.4.2</code> , <code>7.4.4</code>
<code>@redhat-cloud-services/frontend-components</code>	<code>7.7.2</code> , <code>7.7.3</code> , <code>7.7.5</code>
<code>@redhat-cloud-services/rbac-client</code>	<code>9.0.3</code> , <code>9.0.4</code> , <code>9.0.6</code>
<code>@redhat-cloud-services/javascript-clients-shared</code>	<code>2.0.8</code> , <code>2.0.9</code> , <code>2.0.11</code>

Package	Compromised versions
@redhat-cloud-services/frontend-components-config-utilities	4.11.2, 4.11.3, 4.11.5
@redhat-cloud-services/frontend-components-notifications	6.9.2, 6.9.3, 6.9.5
@redhat-cloud-services/tsc-transform-imports	1.2.2, 1.2.4, 1.2.6
@redhat-cloud-services/frontend-components-config	6.11.3, 6.11.4, 6.11.6
@redhat-cloud-services/eslint-config-redhat-cloud-services	3.2.1, 3.2.2, 3.2.4
@redhat-cloud-services/host-inventory-client	5.0.3, 5.0.4, 5.0.6
@redhat-cloud-services/rule-components	4.7.2, 4.7.3, 4.7.5
@redhat-cloud-services/frontend-components-remediations	4.9.2, 4.9.3, 4.9.5
@redhat-cloud-services/frontend-components-translations	4.4.1, 4.4.2, 4.4.4
@redhat-cloud-services/vulnerabilities-client	2.1.9, 2.1.11
@redhat-cloud-services/frontend-components-advisor-components	3.8.2, 3.8.4, 3.8.6
@redhat-cloud-services/entitlements-client	4.0.11, 4.0.12, 4.0.14
@redhat-cloud-services/chrome	2.3.1, 2.3.2, 2.3.4
@redhat-cloud-services/notifications-client	6.1.4, 6.1.5, 6.1.7
@redhat-cloud-services/compliance-client	4.0.3, 4.0.4, 4.0.6
@redhat-cloud-services/sources-client	3.0.10, 3.0.11, 3.0.13
@redhat-cloud-services/integrations-client	6.0.4, 6.0.5, 6.0.7
@redhat-cloud-services/frontend-components-testing	1.2.1, 1.2.2, 1.2.4
@redhat-cloud-services/remediations-client	4.0.4, 4.0.5, 4.0.7
@redhat-cloud-services/insights-client	4.0.4, 4.0.5, 4.0.7
@redhat-cloud-services/topological-inventory-client	3.0.10, 3.0.11, 3.0.13
@redhat-cloud-services/config-manager-client	5.0.4, 5.0.5, 5.0.7
@redhat-cloud-services/hcc-pf-mcp	0.6.1, 0.6.2, 0.6.4
@redhat-cloud-services/quickstarts-client	4.0.11, 4.0.12, 4.0.14
@redhat-cloud-services/patch-client	4.0.4, 4.0.5, 4.0.7
@redhat-cloud-services/hcc-feo-mcp	0.3.1, 0.3.2, 0.3.4
@redhat-cloud-services/hcc-kessel-mcp	0.3.1, 0.3.2, 0.3.4

Hashes reported by Microsoft:

Artifact	Hash
index.js from @redhat-cloud-services/remediations-client	SHA-256 396cac9e457ec54ff6d3f6311cb5cc1da8054d019ce3ffa1de5741506c7a4ea4
index.js from @redhat-cloud-services/frontend-components-advisor-components@3.8.2	SHA-256 d8d170af3de17bb9b217c52aaaffdf9395f35ef015a57ef676e406c121e5e223

Artifact	Hash
<code>index.js</code> from <code>@redhat-cloud-services/hcc-kessel-mcp@0.3.4</code>	SHA-256 f0641e053e81f0d01fa46db35a83e0a34494886503086866d956d14e81fd3e1c
<code>index.js</code> from <code>@redhat-cloud-services/frontend-components-testing@1.2.4</code>	SHA-256 d5a97614d5319ce9c8e01fa0b4eb06fb5b9e54fa13b23d718174a1546444123b
<code>index.js</code> from <code>@redhat-cloud-services/frontend-components-notifications@6.9.3</code>	SHA-256 f88258e21592084a2f93a572ade8f9b91c0cd0e242f5cf6121ed7bad0f7bdd1f
<code>index.js</code> from <code>@redhat-cloud-services/chrome@2.3.4</code>	SHA-256 25e121e3b7d300c0d0075b33e5eca39a3e6a659fb9cfee52b70ef71686628f1b

Axios / Plain-Crypto-JS Compromise (2026)

Summary

On March 31, 2026, malicious Axios versions were published to npm and pulled in `plain-crypto-js@4.2.1`, which executed an install-time payload and downloaded a platform-specific RAT. Microsoft attributed the infrastructure and compromise to Sapphire Sleet.

IoCs

Affected packages:

- `axios@1.14.1`
- `axios@0.30.4`
- `plain-crypto-js@4.2.1`

Known safe downgrade targets from Microsoft guidance:

- `axios@1.14.0`
- `axios@0.30.3`

Network indicators:

- `sfrclak[.]com`
- `142.11.206[.]73`
- `hxxp://sfrclak[.]com:8000/6202033`
- Port `8000` over HTTP

Host indicators:

- `node_modules/plain-crypto-js`
- `node_setup.js` from the `plain-crypto-js` package directory
- `%TEMP%\6202033.vbs`
- `%TEMP%\6202033.ps1`
- `%PROGRAMDATA%\system.bat`
- `C:\ProgramData\wt.exe`
- `/Library/Caches/com.apple.act.mond`
- `/tmp/ld.py`
- `packages.npm.org/product1`
- `packages.npm.org/product0`
- `packages.npm.org/product2`

Hashes reported by Microsoft:

Artifact	Hash
%TEMP%\6202033.ps1	SHA-256 ed8560c1ac7ceb6983ba995124d5917dc1a00288912387a6389296637d5f815c
%TEMP%\6202033.ps1	SHA-256 617b67a8e1210e4fc87c92d1d1da45a2f311c08d26e89b12307cf583c900d101
%PROGRAMDATA%\system.bat	SHA-256 f7d335205b8d7b20208fb3ef93ee6dc817905dc3ae0c10a0b164f4e7d07121cd
/Library/Caches/com.apple.act.mond	SHA-256 92ff08773995ebc8d55ec4b8e1a225d0d1e51efa4ef88b8849d0071230c9645a
/tmp/ld.py	SHA-256 fcb81618bb15edfdedfb638b4c08a2af9cac9ecfa551af135a8402bf980375cf

Ghost Campaign (2026)

Summary

The Ghost campaign used malicious npm packages that displayed fake npm install logs and progress output to hide downloader behavior and phish for a sudo/admin password. The final-stage malware was a RAT designed to steal crypto wallets and sensitive data.

IoCs

Behavioral indicators:

- Fake npm install logs, progress bars, random delays, or bogus dependency download messages.
- Prompt for sudo/admin password during package installation.
- Downloader retrieves payload details from Telegram or web3-themed content.
- Additional **decryptor** file used by some samples.
- Final-stage RAT steals crypto wallets, sensitive data, and accepts C2 commands.

Affected package names and versions reported by ReversingLabs:

Package	Versions
react-performance-suite	2.0.0, 2.0.1
react-state-optimizer-core	1.0.0, 1.0.2-1.0.9, 3.0.3-3.0.9
react-fast-utilisa	2.0.1-2.0.4
ai-fast-auto-trader	2.2.1-2.2.6
pkgnewfefame1	3.2.1
carbon-mac-copy-cloner	1.1.0-1.1.10, 1.2.1
coinbase-desktop-sdk	1.5.14-1.5.17, 1.5.19

Example SHA1 indicators reported by ReversingLabs:

Package version	SHA1
react-performance-suite@2.0.0	bdfffc2f98ff422db9f9ddc190401cfcb686e3c32

Package version	SHA1
react-performance-suite@2.0.1	5928e3121f12f3c5d690bc7968b28b2f67835ef5
react-state-optimizer-core@1.0.0	cbe7c87293de7ab5853e2aef3f638d54c45f5c9f
ai-fast-auto-trader@2.2.1	963b79f59fb2c070a06b9a2af9db2b5512c1ed74
pkgnewfame1@3.2.1	2a8c625660ad6bb7d7c953a147c84c0fcc75794b
carbon-mac-copy-cloner@1.1.0	63783f6e59d20e2c664123b349f22dd53d1293d4
coinbase-desktop-sdk@1.5.17	c02624f8cefe790b6dee529c7a0e97f4241d79ed

PhantomRaven (2025)

Summary

PhantomRaven is an npm malware campaign that abuses Remote Dynamic Dependencies. Instead of showing obvious malicious code in the npm package, packages reference attacker-controlled HTTP URLs so npm fetches and executes the real payload during installation. This can bypass static registry review and dependency graph checks.

IoCs

Behavioral indicators:

- Dependencies that resolve from arbitrary HTTP URLs rather than npm registry versions.
- Lockfile entries containing `http://` package tarball sources.
- Package appears to have zero or trivial dependencies but retrieves additional code during install.
- `preinstall` hook in a fetched remote dependency.
- Exfiltration over HTTP GET, HTTP POST, and WebSocket.
- Harvesting of `.gitconfig`, `.npmrc`, environment variables, GitHub, GitLab, Jenkins, and CircleCI context.

Network and infrastructure indicators reported by Endor Labs:

Wave	Domain	IP	Endpoint	RDD dependency names
Wave 1	packages.storeartifact[.]com	54.173.15[.]59	/jpd.php	ui-styles-pkg
Wave 2	npm.jpartifacts[.]com	100.26.42[.]247	/jpd.php	ui-styles-pkg
Wave 3	package.storeartifacts[.]com	13.219.250[.]107	/npm.php	ui-styles-pkg, js-pkg
Wave 4	npm.artifactsnpm[.]com	54.227.45[.]171	/npm.php	ts-pkg, js-pkg

Additional indicators:

- `http://packages.storeartifact[.]com/npm/`
- `http://packages.storeartifact[.]com/jpd.php`
- `http://npm.jpartifacts[.]com/jpd.php`
- `http://package.storeartifacts[.]com/npm.php`
- `http://npm.artifactsnpm[.]com/npm.php`
- Attacker account email pattern reported in public analysis: `jpdtester01` through `jpdtester13` across free email providers.

Shai-Hulud Worm (2025)

Summary

The original Shai-Hulud campaign was a self-propagating npm worm first publicly reported in September 2025. Malicious npm package versions used post-install scripts to harvest secrets with TruffleHog and other techniques, exfiltrate results to attacker-created GitHub repositories, and publish malicious versions of other packages when npm tokens were found.

IoCs

Package and execution indicators:

- Post-install script added to npm package versions.
- Use of TruffleHog or secret-scanning behavior during package installation.
- Harvesting of environment variables and cloud metadata service credentials.
- Automatic publishing of new malicious npm versions using stolen npm tokens.

Repository indicators:

- Public GitHub repository named [Shai-Hulud](#).
- Secret dump file named [data.json](#).
- Repository description [Shai-Hulud Migration](#).
- Formerly private repositories made public or copied to public repositories.
- Repository names ending in [-migration](#).
- Unexpected workflow files under [.github/workflows/](#).

Network indicator:

- [hxxps://webhook\[.\]site/bb8ca5f6-4175-45d2-b042-fc9ebb8170b7](#)

Credentials to assume exposed:

- GitHub tokens.
- npm tokens.
- SSH keys.
- Environment-variable secrets.
- Atlassian keys.
- Datadog API keys.
- Cloud credentials exposed through local files, environment variables, or IMDS.

References

- Microsoft Security Blog: [Mitigating the Axios npm supply chain compromise](#)
- Microsoft Security Blog: [Preinstall to persistence: Inside the Red Hat npm Miasma credential-stealing campaign](#)
- Wiz: [Miasma: Supply Chain Attack Targeting RedHat npm Packages](#)
- Wiz: [The Worm That Keeps on Digging: TeamPCP Hits @antv in Latest Wave](#)
- Socket: [Mini Shai-Hulud campaign page](#)
- Wiz: [Shai-Hulud npm Supply Chain Attack](#)
- ReversingLabs: [Fake install logs in npm packages load RAT](#)
- Endor Labs: [The Return of PhantomRaven](#)
- Nx: [Postmortem: Nx Console v18.95.0 supply-chain compromise](#)